```
/* ================================================================== */
/* 文件 1/12：projects/offlinelib-core/src/types.ts */
/* 作用：数据结构定义 */
/* ================================================================== */
/**
 * 离线资料库 —— 核心数据结构（接口冻结）
 *
 * WHY THIS FILE EXISTS FIRST
 *   老师对开题的批评里有一条是"全文检索技术路线无法核实"。
 *   核实不了的原因不是想法不行，而是**没有任何可运行的证据**。
 *
 *   所以这个模块的设计原则是：**纯 TypeScript，零依赖，可在 Node 里单测**。
 *   它同时满足三件事：
 *     1. 能在 PC 上跑单测 → 检索路线立刻变成"可核实"
 *     2. 能直接编译进 ArkTS 应用（无 any / 无动态导入 / 无平台 API）
 *     3. 软著登记需要的是"体现软件功能逻辑的源代码"，这本身就是
 *
 *   □□ 本文件**不导入任何 HarmonyOS Kit**。
 *      原因见 docs/11：HMS Kit 在缺少 HSP 的设备上会在**模块加载期**让整个应用崩溃。
 *      核心算法必须与平台解耦，才能既被单测、又不把应用拖死。
 */
/** 一份被导入的资料（一张照片、一个 PDF、一段文本）。 */
export interface DocItem {
  /** 稳定 id，入库后不变。 */
  id: string;
  /** 显示名（文件名或用户命名）。 */
  title: string;
  /** 来源类型。用于界面分组，也用于决定是否需要 OCR。 */
  source: DocSource;
  /** 原始内容的存储位置（URI 或沙箱内路径）。**不存内容本身**，避免重复占空间。 */
  uri: string;
  /** 入库时间（毫秒）。 */
  addedAt: number;
  /** 原文纯文本。OCR 之后填充；纯文本资料直接填充。 */
  text: string;
}
export type DocSource = 'camera' | 'album' | 'file' | 'note';
/**
 * 一个可检索的文本块。
 *
 * 为什么要分块：资料库的文档可能很长，检索需要定位到**具体片段**而不是整篇。
 * 分块策略直接影响召回质量，因此块大小是可调参数。
 */
export interface Chunk {
  /** 全局唯一：`${docId}#${seq}`。 */
  id: string;
  docId: string;
  /** 在原文中的序号。用于"前后文"与高亮定位。 */
  seq: number;
  /** 原文起点（字符偏移）。高亮回原图/原文时用。 */
```

```
  start: number;
  /** 原文终点（不含）。 */
  end: number;
  /** 块文本。 */
  text: string;
}
/**
 * 一条检索命中。
 *
 * 注意 `score` 与 `matched` 是两个不同的东西：
 *   score 是 BM25 相关度，用于排序；
 *   matched 是**具体命中了哪些词**，用于生成高亮与向用户解释"为什么给你这条"。
 */
export interface Hit {
  chunk: Chunk;
  /** BM25 得分。 */
  score: number;
  /**
   * 命中的词项 → 该块中的出现次数。
   *
   * 用 `Map` 而不是 `Record<string, number>`：ArkTS 拒绝给索引签名类型赋对象字面量
   * （`arkts-no-untyped-obj-literals`），而 `Map` 在 TS 与 ArkTS 下行为一致。
   */
  matched: Map<string, number>;
  /** 高亮区间（相对 chunk.text 的字符偏移）。 */
  spans: Span[];
}
/** 高亮区间，左闭右开。 */
export interface Span {
  start: number;
  end: number;
}
/** 检索选项。`topK` 由预算决策给出，不在这里写死。 */
export interface SearchOptions {
  /** 返回多少条。<=0 表示由得分分布自适应决定。 */
  topK: number;
  /** 最少返回几条（自适应时作为下界）。 */
  minK: number;
  /** 最多返回几条（自适应时作为上界）。 */
  maxK: number;
}
/** 检索结果，含决策依据，便于向用户解释。 */
export interface SearchResult {
  hits: Hit[];
  /** 实际使用的 k。 */
  k: number;
  /** k 是怎么定的：'fixed' | 'score-gap'。 */
  kSource: 'fixed' | 'score-gap';
  /** 用于决策的得分统计（自适应时为断层信息）。 */
  diagnostics: ScoreDiagnostics;
```

```
}
export interface ScoreDiagnostics {
  /** 命中的候选数（未截断前）。 */
  candidateCount: number;
  /** 最高分。 */
  topScore: number;
  /** 全部候选得分的均值和标准差。 */
  mean: number;
  std: number;
  /** 相邻得分之间的最大相对落差，以及它出现在第几位。 */
  maxGap: number;
  maxGapAt: number;
}
/** 预算决策的输入。字段都在手机上有真实含义，不是抽象参数。 */
export interface BudgetInput {
  /** 可用于检索的时间预算（毫秒）。 */
  latencyBudgetMs: number;
  /** 可用于本次检索的内存预算（KB）。 */
  memoryBudgetKb: number;
  /** 当前电量百分比（0-100）。低于阈值时降级。 */
  batteryPercent: number;
}
/** 预算决策的输出：这一次该取几条。 */
export interface BudgetDecision {
  k: number;
  /** 决策理由，直接显示给用户（"为什么只给你 3 条"）。 */
  reason: string;
}
/**
 * 持久化存档格式。
 *
 * 只存文档，不存倒排索引——理由见 `store.ts` 顶部的 DESIGN DECISION。
 * `version` 用于将来做格式迁移；目前只支持 1。
 */
export interface DocStoreSnapshot {
  version: number;
  docs: DocItem[];
}
/* ========================================================================== */
/* 文件 2/12：projects/offlinelib-core/src/tokenize.ts */
/* 作用：中文单字与二元组切词 */
/* ========================================================================== */
/**
 * 切词 —— 不依赖任何词典。
 *
 * WHY NO DICTIONARY
 *   手机上没有中文分词词典，也不该为一个个人资料库塞一个几十 MB 的词表。
 *   所以用**二元组（bigram）**：
 *     "离线资料库" → 离/线/资/料/库 + 离线/线资/资料/料库
 *
```

```
 *   这是中文检索里的经典做法：不用分词也能达到可用的召回率，
 *   代价是索引会大一些（约为字数的 2 倍词项）。
 *
 *   英文与数字按非字母数字切分并转小写。
 *
 * □□ 纯函数，无副作用，无平台 API —— 因此可直接在 Node 里单测。
 */
/** 一个词项及其在文本中的位置（用于高亮）。 */
export interface Token {
  term: string;
  start: number;
  end: number;
}
const CJK_START = 0x4e00;
const CJK_END = 0x9fff;
const CJK_EXT_START = 0x3400;
const CJK_EXT_END = 0x4dbf;
export function isCJK(ch: string): boolean {
  const c = ch.charCodeAt(0);
  return (c >= CJK_START && c <= CJK_END) || (c >= CJK_EXT_START && c <= CJK_EXT_END);
}
function isAlnum(ch: string): boolean {
  return /[0-9a-z]/.test(ch);
}
/**
 * 切词。
 *
 * 中文：单字 + 相邻二元组
 * 英文/数字：连续字母数字串，转小写
 *
 * @param text 原文
 * @param keepUnigram 是否保留中文单字。默认 true。
 *                    设为 false 可让索引小约一半，但"单字查询"会失效。
 */
export function tokenize(text: string, keepUnigram: boolean = true): Token[] {
  const out: Token[] = [];
  const lower = text.toLowerCase();
  const n = lower.length;
  let i = 0;
  while (i < n) {
    const ch = lower.charAt(i);
    if (isCJK(ch)) {
      // 单字
      if (keepUnigram) {
        out.push({ term: ch, start: i, end: i + 1 });
      }
      // 二元组（需要下一个字符也是 CJK）
      if (i + 1 < n && isCJK(lower.charAt(i + 1))) {
        out.push({ term: lower.substring(i, i + 2), start: i, end: i + 2 });
      }
```

```
      i += 1;
      continue;
    }
    if (isAlnum(ch)) {
      let j = i;
      while (j < n && isAlnum(lower.charAt(j))) {
        j += 1;
      }
      const term = lower.substring(i, j);
      out.push({ term: term, start: i, end: j });
      i = j;
      continue;
    }
    // 其他字符（标点、空白、emoji）不产生词项
    i += 1;
  }
  return out;
}
/** 只要词项、不要位置时用它（建索引时用）。 */
export function terms(text: string, keepUnigram: boolean = true): string[] {
  const toks = tokenize(text, keepUnigram);
  const out: string[] = [];
  for (let i = 0; i < toks.length; i++) {
    out.push(toks[i].term);
  }
  return out;
}
/* ========================================================================== */
/* 文件 3/12：projects/offlinelib-core/src/index.ts */
/* 作用：倒排索引与 BM25 打分 */
/* ========================================================================== */
/**
 * 倒排索引 + BM25 打分 + 分块。
 *
 * WHY BM25 AND NOT VECTORS
 *   - 向量检索在手机上要么依赖 HMS 的 Retrieval（我们无法验证它是否存在），
 *     要么自己跑一个嵌入模型（手机上没有可用的端侧大模型）。
 *   - BM25 是**纯统计**方法：零依赖、确定性、可在 Node 里逐条断言。
 *   对一个"个人资料库"来说，BM25 的召回质量足够，而且它的打分分布
 *   **恰好就是自适应 top-k 要用的信号**（见 search.ts）。
 *
 * □□ 纯计算，不导入任何平台 API。
 */
import { Chunk, DocItem, Span } from './types';
import { terms, tokenize } from './tokenize';
/** BM25 参数。默认值是文献里的常用取值。 */
export const BM25_K1 = 1.5;
export const BM25_B = 0.75;
/** 分块参数。 */
export interface ChunkOptions {
```

```
  /** 每块目标字符数。中文按字符算，英文按字符算（不做语言区分，保持简单）。 */
  size: number;
  /** 相邻块的重叠字符数。避免关键词被切在边界上而漏检。 */
  overlap: number;
}
export const DEFAULT_CHUNK_OPTIONS: ChunkOptions = { size: 240, overlap: 40 };
/** 倒排表的一项：某个词项 → (块 id → 词频)。 */
interface Posting {
  tf: number;
  positions: number[];
}
export class InvertedIndex {
  /** 全部块，按加入顺序。**含已删除的墓碑**，所以不能直接用下标当"第 i 个活块"。 */
  private chunks: Chunk[] = [];
  /** 块 id → 下标。 */
  private chunkPos: Map<string, number> = new Map<string, number>();
  /** 词项 → (块下标 → Posting)。 */
  private postings: Map<string, Map<number, Posting>> = new Map<string, Map<number, Posting>>();
  /** 每个块的长度（词项数）。与 chunks 等长。 */
  private lengths: number[] = [];
  /** 墓碑标记。true 表示该块已删除，检索时跳过。 */
  private deleted: boolean[] = [];
  /** 活块的物理下标列表，顺序与加入顺序一致。`getChunk(i)` 用它映射。 */
  private liveIdx: number[] = [];
  /** 活块数。 */
  private liveCount: number = 0;
  /** 活块的长度总和，用于 avgdl。 */
  private liveLengthSum: number = 0;
  /**
   * 墓碑超过这个比例就自动压缩。
   *
   * 为什么需要：墓碑让删除变成 O(文档块数) 而不是 O(全库)，
   * 但死块会一直占着内存与遍历成本。20% 是一个折中——
   * 压缩本身是 O(全库)，太频繁就失去了增量的意义。
   */
  public static readonly COMPACT_THRESHOLD = 0.2;
  /** 已索引的**活块**数。 */
  public size(): number {
    return this.liveCount;
  }
  /** 清空。重新建库时用。 */
  public clear(): void {
    this.chunks = [];
    this.chunkPos = new Map<string, number>();
    this.postings = new Map<string, Map<number, Posting>>();
    this.lengths = [];
    this.deleted = [];
    this.liveIdx = [];
    this.liveCount = 0;
    this.liveLengthSum = 0;
```

```
}
/** 加入一个块。重复 id 会被忽略（幂等，便于增量建库）。 */
public addChunk(c: Chunk): boolean {
  if (this.chunkPos.has(c.id)) {
    return false;
  }
  const idx = this.chunks.length;
  this.chunks.push(c);
  this.chunkPos.set(c.id, idx);
  this.deleted.push(false);
  this.liveIdx.push(idx);
  const toks = tokenize(c.text);
  this.lengths.push(toks.length);
  this.liveCount += 1;
  this.liveLengthSum += toks.length;
  // 逐词项累积词频与位置
  for (let i = 0; i < toks.length; i++) {
    const t = toks[i];
    let perChunk = this.postings.get(t.term);
    if (perChunk === undefined) {
      perChunk = new Map<number, Posting>();
      this.postings.set(t.term, perChunk);
    }
    let p = perChunk.get(idx);
    if (p === undefined) {
      p = { tf: 0, positions: [] };
      perChunk.set(idx, p);
    }
    p.tf += 1;
    p.positions.push(t.start);
  }
  return true;
}
/**
 * 删除一份文档的全部块。
 *
 * 用**墓碑标记**而不是物理删除：物理删除会让 `chunks` 下标整体前移，
 * 所有倒排表里记录的块下标立刻全部失效，只能全量重建——
 * 那正是我们要避免的。墓碑让删除只与**该文档的块数**成正比。
 *
 * 返回被标记删除的块数。
 */
public removeDocument(docId: string): number {
  let removed = 0;
  for (let i = 0; i < this.liveIdx.length; i++) {
    const phys = this.liveIdx[i];
    if (this.chunks[phys].docId !== docId) {
      continue;
    }
    this.deleted[phys] = true;
```

```
    this.liveCount -= 1;
    this.liveLengthSum -= this.lengths[phys];
    removed += 1;
  }
  if (removed > 0) {
    this.rebuildLiveIdx();
    this.maybeCompact();
  }
  return removed;
}
/**
 * 用新文档替换旧文档的同 id 部分。
 *
 * 这是"用户重拍一张更清晰的照片"对应的操作：
 * 旧块墓碑化，新块加入。比"先删再建索引"少一次全量重建。
 *
 * 返回新加入的块数。
 */
public replaceDocument(doc: DocItem, opts: ChunkOptions = DEFAULT_CHUNK_OPTIONS): number {
  this.removeDocument(doc.id);
  // 清掉该 docId 的**全部**块 id 占位。
  //
  // □□ 这里曾经只遍历 liveIdx，结果漏掉了已墓碑化的旧块——
  //    它们的 id 仍留在 chunkPos 里，于是新块被当成"重复 id"而拒绝加入，
  //    表现为"替换后搜不到新内容"。
  //
  //    为什么早期测试没发现：那些用例是单文档库，删除后墓碑比例 100%
  //    会触发自动压缩，压缩顺带重建了 chunkPos，把 bug 掩盖了。
  //    多文档场景（墓碑比例低于阈值、不触发压缩）才暴露出来。
  for (let i = 0; i < this.chunks.length; i++) {
    if (this.chunks[i].docId === doc.id) {
      this.chunkPos.delete(this.chunks[i].id);
    }
  }
  return this.addDocument(doc, opts);
}
/** 活块的物理下标重排。删除后调用。 */
private rebuildLiveIdx(): void {
  const next: number[] = [];
  for (let i = 0; i < this.chunks.length; i++) {
    if (!this.deleted[i]) {
      next.push(i);
    }
  }
  this.liveIdx = next;
}
/** 墓碑比例超阈值时自动压缩。 */
private maybeCompact(): void {
  const total = this.chunks.length;
  if (total === 0) {
```

```
    return;
  }
  const deadRatio = (total - this.liveCount) / total;
  if (deadRatio >= InvertedIndex.COMPACT_THRESHOLD) {
    this.compact();
  }
}
/**
 * 物理压缩：丢掉墓碑，重建全部倒排表。
 *
 * **这是 O(全库) 的操作**，所以只在墓碑比例高时调用——
 * 频繁调用会让增量删除失去意义。
 */
public compact(): void {
  const keptChunks: Chunk[] = [];
  const keptLengths: number[] = [];
  const remap: Map<number, number> = new Map<number, number>();
  for (let i = 0; i < this.chunks.length; i++) {
    if (this.deleted[i]) {
      continue;
    }
    remap.set(i, keptChunks.length);
    keptChunks.push(this.chunks[i]);
    keptLengths.push(this.lengths[i]);
  }
  this.chunks = keptChunks;
  this.lengths = keptLengths;
  this.deleted = [];
  for (let i = 0; i < keptChunks.length; i++) {
    this.deleted.push(false);
  }
  this.liveIdx = [];
  for (let i = 0; i < keptChunks.length; i++) {
    this.liveIdx.push(i);
  }
  this.liveCount = keptChunks.length;
  this.chunkPos = new Map<string, number>();
  for (let i = 0; i < keptChunks.length; i++) {
    this.chunkPos.set(keptChunks[i].id, i);
  }
  // 重建倒排表：把旧下标重映射到新下标
  const nextPostings: Map<string, Map<number, Posting>> = new Map<string, Map<number, Posting>>();
  this.postings.forEach((perChunk: Map<number, Posting>, term: string) => {
    const remapped: Map<number, Posting> = new Map<number, Posting>();
    perChunk.forEach((p: Posting, oldIdx: number) => {
      const ni = remap.get(oldIdx);
      if (ni !== undefined) {
        remapped.set(ni, p);
      }
    });
```

```
    if (remapped.size > 0) {
      nextPostings.set(term, remapped);
    }
  });
  this.postings = nextPostings;
}
/** 墓碑数。用于界面上显示"待整理"或调试。 */
public tombstoneCount(): number {
  return this.chunks.length - this.liveCount;
}
/** 把一份资料切成块并全部加入。返回新建的块数。 */
public addDocument(doc: DocItem, opts: ChunkOptions = DEFAULT_CHUNK_OPTIONS): number {
  const chunks = chunkDocument(doc, opts);
  let added = 0;
  for (let i = 0; i < chunks.length; i++) {
    if (this.addChunk(chunks[i])) {
      added += 1;
    }
  }
  return added;
}
/** 平均块长度（词项数）。只统计活块——墓碑不该影响 avgdl。 */
public avgDocLength(): number {
  if (this.liveCount === 0) {
    return 0;
  }
  return this.liveLengthSum / this.liveCount;
}
/** 活块数。 */
public chunkCount(): number {
  return this.liveCount;
}
/**
 * 取第 i 个**活块**。
 *
 * 注意入参是"活块序号"，不是物理下标——真实下标由 `liveIdx` 映射。
 * 这样调用方（分块遍历、高亮、界面列表）不需要知道墓碑的存在。
 */
public getChunk(i: number): Chunk {
  return this.chunks[this.liveIdx[i]];
}
/**
 * BM25 打分。
 *
 * 返回**全部候选**（未截断）的得分，按降序排。
 * 不截断是刻意的：自适应 top-k 需要看完整分布才能找断层（见 search.ts）。
 */
public score(query: string): Candidate[] {
  const qTerms = terms(query);
  if (qTerms.length === 0 || this.chunks.length === 0) {
```

```
  return [];
}
const N = this.chunks.length;
const avgdl = this.avgDocLength();
// 同一查询里重复的词项只计一次
const uniq: string[] = [];
const seen: Map<string, boolean> = new Map<string, boolean>();
for (let i = 0; i < qTerms.length; i++) {
  if (!seen.has(qTerms[i])) {
    seen.set(qTerms[i], true);
    uniq.push(qTerms[i]);
  }
}
const acc: Map<number, number> = new Map<number, number>();
const matched: Map<number, Map<string, number>> = new Map<number, Map<string, number>>();
for (let qi = 0; qi < uniq.length; qi++) {
  const term = uniq[qi];
  const perChunk = this.postings.get(term);
  if (perChunk === undefined) {
    continue;
  }
  const df = perChunk.size;
  // BM25 的 IDF，加 1 保证非负
  const idf = Math.log((N - df + 0.5) / (df + 0.5) + 1);
  perChunk.forEach((p: Posting, idx: number) => {
    const dl = this.lengths[idx];
    const denom = p.tf + BM25_K1 * (1 - BM25_B + (BM25_B * dl) / avgdl);
    const gain = (idf * (p.tf * (BM25_K1 + 1))) / denom;
    acc.set(idx, (acc.get(idx) ?? 0) + gain);
    let m = matched.get(idx);
    if (m === undefined) {
      m = new Map<string, number>();
      matched.set(idx, m);
    }
    m.set(term, p.tf);
  });
}
const out: Candidate[] = [];
acc.forEach((score: number, idx: number) => {
  // 墓碑块必须跳过：它们已从 liveCount / avgdl 里剔除，
  // 但倒排表里仍留着记录（要等 compact 才清）。
  if (this.deleted[idx]) {
    return;
  }
  const m = matched.get(idx);
  out.push({
    index: idx,
    chunk: this.chunks[idx],
    score: score,
    matched: m !== undefined ? m : new Map<string, number>()
```

```
      });
    });
    out.sort((a: Candidate, b: Candidate) => b.score - a.score);
    return out;
  }
}
/** 一个候选（未截断）。 */
export interface Candidate {
  /** 块在索引里的下标。 */
  index: number;
  chunk: Chunk;
  score: number;
  matched: Map<string, number>;
}
/**
 * 把一份资料切成重叠的块。
 *
 * 重叠是必需的：中文关键词被切在块边界上会导致漏检，
 * 而 40 字符的重叠足以覆盖绝大多数被切断的词。
 */
export function chunkDocument(doc: DocItem, opts: ChunkOptions = DEFAULT_CHUNK_OPTIONS): Chunk[] {
  const text = doc.text;
  const out: Chunk[] = [];
  if (text.length === 0) {
    return out;
  }
  const size = Math.max(1, opts.size);
  const overlap = Math.max(0, Math.min(opts.overlap, size - 1));
  const step = Math.max(1, size - overlap);
  let seq = 0;
  let start = 0;
  while (start < text.length) {
    const end = Math.min(start + size, text.length);
    out.push({
      id: doc.id + '#' + seq.toString(),
      docId: doc.id,
      seq: seq,
      start: start,
      end: end,
      text: text.substring(start, end)
    });
    seq += 1;
    if (end >= text.length) {
      break;
    }
    start += step;
  }
  return out;
}
/** 在块内定位查询词的高亮区间。 */
```

```
export function locateSpans(chunkText: string, matched: Map<string, number>): Span[] {
  const spans: Span[] = [];
  const lower = chunkText.toLowerCase();
  matched.forEach((_tf: number, term: string) => {
    let from = 0;
    while (true) {
      const at = lower.indexOf(term, from);
      if (at < 0) {
        break;
      }
      spans.push({ start: at, end: at + term.length });
      from = at + term.length;
    }
  });
  // 按起点排序并合并重叠区间，避免界面上重复绘制
  spans.sort((a: Span, b: Span) => a.start - b.start);
  return mergeSpans(spans);
}
function mergeSpans(spans: Span[]): Span[] {
  if (spans.length <= 1) {
    return spans;
  }
  const out: Span[] = [];
  let cur = spans[0];
  for (let i = 1; i < spans.length; i++) {
    const s = spans[i];
    if (s.start <= cur.end) {
      cur = { start: cur.start, end: Math.max(cur.end, s.end) };
    } else {
      out.push(cur);
      cur = s;
    }
  }
  out.push(cur);
  return out;
}
/* ============================================================================ */
/* 文件 4/12：projects/offlinelib-core/src/search.ts */
/* 作用：检索流程与自适应 k */
/* ============================================================================ */
/**
 * 检索 + 自适应 top-k 决策。
 *
 * THIS IS THE PART THAT CARRIES THE RESEARCH
 *   我们的调度器实验得出的结论是：
 *     **"剩余预算"不是决定检索深度的正确信号；"候选得分分布"才是。**
 *
 *   在资料库这个场景里，这句话变得非常具体：
 *     得分分布有明显断层 → 断层之前的是真正相关的，之后就不要再取；
 *     得分分布平坦       → 没有明显区分度，多取无益。
```

```
 *
 *   所以 `kSource: 'score-gap'` 这条路径，就是那个研究结论的产品化实现。
 *   而 `kSource: 'fixed'` 是它的对照组 —— 产品里两者都要保留，
 *   因为**用户能直观看到"固定 k"和"自适应 k"的差别**，这本身就是产品价值。
 *
 * □□ 纯函数，可在 Node 里逐条断言。
 */
import { InvertedIndex, Candidate, locateSpans } from './index';
import {
  Hit, ScoreDiagnostics, SearchOptions, SearchResult, BudgetInput, BudgetDecision
} from './types';
/** 相对落差的下限：小于这个值的"断层"不算断层（避免把噪声当信号）。 */
export const MIN_GAP_RATIO = 0.05;
/**
 * 计算得分分布诊断。
 *
 * `maxGap` 用**相对落差**而不是绝对差：不同查询的绝对得分量级不同，
 * 只有相对值才可比。
 */
export function diagnose(candidates: Candidate[]): ScoreDiagnostics {
  const n = candidates.length;
  if (n === 0) {
    return { candidateCount: 0, topScore: 0, mean: 0, std: 0, maxGap: 0, maxGapAt: -1 };
  }
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += candidates[i].score;
  }
  const mean = sum / n;
  let sq = 0;
  for (let i = 0; i < n; i++) {
    const d = candidates[i].score - mean;
    sq += d * d;
  }
  const std = Math.sqrt(sq / n);
  let maxGap = 0;
  let maxGapAt = -1;
  for (let i = 0; i + 1 < n; i++) {
    const hi = candidates[i].score;
    const lo = candidates[i + 1].score;
    if (hi <= 0) {
      continue;
    }
    const ratio = (hi - lo) / hi;
    if (ratio > maxGap) {
      maxGap = ratio;
      maxGapAt = i;
    }
  }
  return {
```

```
    candidateCount: n,
    topScore: candidates[0].score,
    mean: mean,
    std: std,
    maxGap: maxGap,
    maxGapAt: maxGapAt
  };
}
/**
 * 由得分分布决定 k。
 *
 * 规则：
 *   1. 找到相邻得分的最大相对落差位置 g
 *   2. 若该落差 >= MIN_GAP_RATIO，取 k = g + 1（断层之前的全部保留）
 *   3. 否则说明分布平坦、没有明显相关边界 → 取 maxK（反正区分不出来）
 *   4. 最后夹到 [minK, maxK] 与候选总数内
 */
export function decideKByScoreGap(
  candidates: Candidate[],
  minK: number,
  maxK: number,
  diag?: ScoreDiagnostics
): number {
  const n = candidates.length;
  if (n === 0) {
    return 0;
  }
  const d = diag !== undefined ? diag : diagnose(candidates);
  let k: number;
  if (d.maxGapAt >= 0 && d.maxGap >= MIN_GAP_RATIO) {
    k = d.maxGapAt + 1;
  } else {
    // 分布平坦：没有可信的断层，用上界
    k = maxK;
  }
  if (k < minK) {
    k = minK;
  }
  if (k > maxK) {
    k = maxK;
  }
  if (k > n) {
    k = n;
  }
  return k;
}
/**
 * 检索主入口。
 *
 * `options.topK > 0` → 固定 k（对照组）
```

```
 * `options.topK <= 0` → 由得分分布自适应（我们的方法）
 */
export function search(index: InvertedIndex, query: string, options: SearchOptions): SearchResult {
  const candidates = index.score(query);
  const diag = diagnose(candidates);
  let k: number;
  let kSource: 'fixed' | 'score-gap';
  if (options.topK > 0) {
    k = options.topK;
    kSource = 'fixed';
  } else {
    k = decideKByScoreGap(candidates, options.minK, options.maxK, diag);
    kSource = 'score-gap';
  }
  if (k > candidates.length) {
    k = candidates.length;
  }
  const hits: Hit[] = [];
  for (let i = 0; i < k; i++) {
    const c = candidates[i];
    hits.push({
      chunk: c.chunk,
      score: c.score,
      matched: c.matched,
      spans: locateSpans(c.chunk.text, c.matched)
    });
  }
  return { hits: hits, k: k, kSource: kSource, diagnostics: diag };
}
/**
 * 由资源预算决定"还能不能多取几条"。
 *
 * 注意这与上面的 k 决策是**两件事**：
 *   - `decideKByScoreGap` 回答"该取几条"（信息层面的问题，看得分分布）
 *   - `decideBudget`    回答"还取得起吗"（资源层面的问题，看延迟/内存/电量）
 *
 * 这正是我们研究主张的另一半：**两个问题需要两个不同的信号**。
 * 把它们混成一个"剩余预算"信号，就会得到实验里那个被严格支配的结果。
 */
export function decideBudget(input: BudgetInput, avgChunkChars: number): BudgetDecision {
  // 每条命中大约要一次排序 + 一次高亮定位，按经验折算成时间
  const msPerHit = 0.6;
  const kbPerHit = (avgChunkChars * 2) / 1024 + 0.5;
  const byLatency = Math.floor(input.latencyBudgetMs / msPerHit);
  const byMemory = Math.floor(input.memoryBudgetKb / kbPerHit);
  let k = Math.min(byLatency, byMemory);
  const reasons: string[] = [];
  if (byLatency <= byMemory) {
    reasons.push('受时间预算限制');
  } else {
```

```
    reasons.push('受内存预算限制');
  }
  // 低电量：主动降级，避免后台建库/检索耗电
  if (input.batteryPercent <= 15) {
    k = Math.floor(k / 2);
    reasons.push('电量偏低，已降级');
  }
  if (k < 1) {
    k = 1;
    reasons.push('已压到最小 1 条');
  }
  return { k: k, reason: reasons.join('；') };
}
/* ========================================================================== */
/* 文件 5/12：projects/offlinelib-core/src/store.ts */
/* 作用：文档存储与持久化 */
/* ========================================================================== */
/**
 * 文档存储与持久化。
 *
 * DESIGN DECISION: 存文档，不存索引
 *   另一种做法是把倒排表序列化下来（省启动时间），但代价是：
 *   索引格式一旦改动，旧存档就废了；而且序列化/反序列化本身容易出静默错误
 *   （位置数组、Map 的键类型、浮点精度）。
 *
 *   个人资料库的文档量级（几百到几千）下，**重建索引是秒级**的。
 *   所以这里选择：**只持久化 DocItem[]，启动时重建索引。**
 *
 *   这条取舍写在代码里而不是只写在文档里——因为半年后改代码的人
 *   会先看这里，而不是看 README。
 *
 * □□ 纯 TS：不触碰文件系统。真正的读写由调用方（ArkTS 层的 preferences/文件 API）
 *    完成，本模块只负责"文档列表 □ JSON 字符串"这一段。
 *    这样它才能既被 Node 单测、又不在缺少平台能力时崩溃。
 */
import { DocItem, DocSource, DocStoreSnapshot } from './types';
/** 当前存档格式版本。格式变更时递增，load 时按版本迁移。 */
export const SNAPSHOT_VERSION = 1;
/**
 * 文档库。
 *
 * 只负责增删查与序列化，**不做索引**——索引由 InvertedIndex 管。
 * 两者分开的原因：文档是"数据"，索引是"数据之上的派生结构"，
 * 混在一起会让"重建索引"和"改文档"难以区分。
 */
export class DocStore {
  private docs: DocItem[] = [];
  private byId: Map<string, DocItem> = new Map<string, DocItem>();
  /** 全部文档（副本，避免外部改内部状态）。 */
  public all(): DocItem[] {
```

```
  const out: DocItem[] = [];
  for (let i = 0; i < this.docs.length; i++) {
    out.push(this.docs[i]);
  }
  return out;
}
public size(): number {
  return this.docs.length;
}
public get(id: string): DocItem | undefined {
  return this.byId.get(id);
}
public has(id: string): boolean {
  return this.byId.has(id);
}
/**
 * 加入一份文档。id 已存在时**覆盖**（并保持原位置），返回是否为新文档。
 *
 * 覆盖而不是拒绝：OCR 可能被重跑（用户重拍了一张更清晰的照片），
 * 这时替换文本是合理行为，不该报错。
 */
public put(doc: DocItem): boolean {
  const isNew = !this.byId.has(doc.id);
  this.byId.set(doc.id, doc);
  if (isNew) {
    this.docs.push(doc);
  } else {
    for (let i = 0; i < this.docs.length; i++) {
      if (this.docs[i].id === doc.id) {
        this.docs[i] = doc;
        break;
      }
    }
  }
  return isNew;
}
/** 删除。返回是否真的删掉了。 */
public remove(id: string): boolean {
  if (!this.byId.has(id)) {
    return false;
  }
  this.byId.delete(id);
  const next: DocItem[] = [];
  for (let i = 0; i < this.docs.length; i++) {
    if (this.docs[i].id !== id) {
      next.push(this.docs[i]);
    }
  }
  this.docs = next;
  return true;
```

```
  }
  public clear(): void {
    this.docs = [];
    this.byId = new Map<string, DocItem>();
  }
  /** 序列化为 JSON 字符串。 */
  public save(): string {
    const snap: DocStoreSnapshot = { version: SNAPSHOT_VERSION, docs: this.all() };
    return JSON.stringify(snap);
  }
  /**
   * 从 JSON 字符串恢复。
   *
   * **容错策略**：跳过不合法的条目，而不是整体失败。
   * 理由：一份存档里坏了一条，不该让用户丢掉其余全部资料。
   * 返回值是"被跳过的条数"，调用方可以据此提示用户。
   */
  public load(json: string): number {
    this.clear();
    if (json.length === 0) {
      return 0;
    }
    let raw: DocStoreSnapshot;
    try {
      raw = JSON.parse(json) as DocStoreSnapshot;
    } catch (e) {
      return 0;
    }
    if (raw === null || raw === undefined || raw.docs === undefined) {
      return 0;
    }
    let skipped = 0;
    for (let i = 0; i < raw.docs.length; i++) {
      const d = raw.docs[i];
      if (!isValidDoc(d)) {
        skipped += 1;
        continue;
      }
      this.put(d);
    }
    return skipped;
  }
}
/** 校验一个从 JSON 读进来的对象是否是可用的 DocItem。 */
export function isValidDoc(d: DocItem): boolean {
  if (d === null || d === undefined) {
    return false;
  }
  if (typeof d.id !== 'string' || d.id.length === 0) {
    return false;
```

```
  }
  if (typeof d.title !== 'string') {
    return false;
  }
  if (typeof d.text !== 'string') {
    return false;
  }
  if (typeof d.addedAt !== 'number' || !isFinite(d.addedAt)) {
    return false;
  }
  if (!isValidSource(d.source)) {
    return false;
  }
  return true;
}
export function isValidSource(s: DocSource): boolean {
  return s === 'camera' || s === 'album' || s === 'file' || s === 'note';
}
/** 生成一个文档 id。时间戳 + 序号，够用且可读。 */
export function makeDocId(seq: number, now: number): string {
  return 'd' + now.toString(36) + '-' + seq.toString(36);
}
/* ========================================================================== */
/* 文件 6/12：projects/offlinelib-core/src/ocr.ts */
/* 作用：OCR 抽象层与质量判定 */
/* ========================================================================== */
/**
 * OCR 抽象层。
 *
 * WHY AN INTERFACE INSTEAD OF CALLING THE KIT DIRECTLY
 *   这是整个工程最重要的一条架构决定。
 *
 *   我们实测过一个真实的崩溃：应用静态 `import` 了 HMS 的 A2A Kit，
 *   而设备上没有对应的 HSP 时，ArkTS 运行时会在**模块加载期**直接
 *   `abort` 整个进程 —— 不是"功能不可用"，是"应用起不来"。
 *   （崩溃日志：`load hsp failed, hsp name: com.huawei.hmsapp.hiai.hsp/agentKitHsp`）
 *
 *   所以 OCR 在这里是一个**接口**，而不是一个 import。
 *   真正的实现（调用 Vision Kit）由上层注入，并且**必须在使用前先探测能力是否存在**。
 *   这样：
 *     1. 本模块可以在 Node 里被单测，不需要设备
 *     2. 设备上没有 OCR 时，应用仍然能启动（只是这个能力降级）
 *
 * □□ 本文件不导入任何 HarmonyOS Kit。这是刻意的。
 */
/** 一次 OCR 的结果。 */
export interface OcrResult {
  /** 识别出的纯文本。失败时为空串。 */
  text: string;
  /** 平均置信度 0-1。0 表示完全不可信或识别失败。 */
```

```
  confidence: number;
  /** 用时（毫秒），用于预算决策。 */
  elapsedMs: number;
  /** 失败原因；成功时为 undefined。 */
  error?: string;
}
/** OCR 引擎。真实实现由 ArkTS 侧提供。 */
export interface OcrEngine {
  /** 引擎名，用于界面显示与排错。 */
  name(): string;
  /** 这个引擎在当前设备上是否可用。**调用方必须先问这个，再决定用不用。** */
  available(): boolean;
  /** 识别一张图片。 */
  recognize(imageUri: string): Promise<OcrResult>;
}
/**
 * 假 OCR 引擎 —— 用预设文本代替真实识别。
 *
 * 用途有三个，都不是"应付测试"：
 *   1. **让检索流水线在没有设备的机器上可端到端跑通**（老师要的"可核实"）
 *   2. 在设备 OCR 不可用时作为**降级占位**，让其余功能仍可演示
 *   3. 造可控的"识别质量差"场景，用来测调度模块的降级逻辑
 */
export class FakeOcrEngine implements OcrEngine {
  private table: Map<string, OcrResult>;
  private defaultText: string;
  private failUris: Set<string>;
  /**
   * @param defaultText 没有预设时返回的文本
   */
  constructor(defaultText: string = '') {
    this.table = new Map<string, OcrResult>();
    this.defaultText = defaultText;
    this.failUris = new Set<string>();
  }
  public name(): string {
    return 'fake-ocr';
  }
  public available(): boolean {
    return true;
  }
  /** 预设某个 URI 的识别结果。 */
  public preset(uri: string, text: string, confidence: number): FakeOcrEngine {
    this.table.set(uri, { text: text, confidence: confidence, elapsedMs: 10 });
    return this;
  }
  /** 预设某个 URI 识别失败。 */
  public presetFailure(uri: string): FakeOcrEngine {
    this.failUris.add(uri);
    return this;
```

```
  }
  public recognize(imageUri: string): Promise<OcrResult> {
    if (this.failUris.has(imageUri)) {
      return Promise.resolve({
        text: '',
        confidence: 0,
        elapsedMs: 5,
        error: 'fake: 预设的识别失败'
      });
    }
    const hit = this.table.get(imageUri);
    if (hit !== undefined) {
      return Promise.resolve(hit);
    }
    return Promise.resolve({
      text: this.defaultText,
      confidence: this.defaultText.length > 0 ? 0.5 : 0,
      elapsedMs: 5,
      error: this.defaultText.length > 0 ? undefined : 'fake: 无预设文本'
    });
  }
}
/**
 * 一个永远不可用的引擎 —— 用于模拟"设备没有 OCR 能力"。
 *
 * 它的存在是为了让"能力缺失时应用仍能启动"这条设计**可被测试**,
 * 而不是只在文档里承诺。
 */
export class UnavailableOcrEngine implements OcrEngine {
  private why: string;
  constructor(why: string = '设备不具备 OCR 能力') {
    this.why = why;
  }
  public name(): string {
    return 'unavailable-ocr';
  }
  public available(): boolean {
    return false;
  }
  public recognize(_imageUri: string): Promise<OcrResult> {
    return Promise.resolve({
      text: '',
      confidence: 0,
      elapsedMs: 0,
      error: this.why
    });
  }
}
/**
 * 根据识别置信度决定"要不要提示用户重拍"。
```

```
 *
 * 这是三个信号里的**感知环节信号**：图像质量 → 是否重拍。
 * 对应研究主张里"感知（OCR）应当由图像质量驱动"这一条。
 */
export interface OcrQualityPolicy {
  /** 低于这个置信度就提示重拍。 */
  retakeThreshold: number;
  /** 低于这个置信度就认为识别失败，不应入库。 */
  rejectThreshold: number;
}
export const DEFAULT_OCR_POLICY: OcrQualityPolicy = {
  retakeThreshold: 0.6,
  rejectThreshold: 0.2
};
export type OcrVerdict = 'accept' | 'retake' | 'reject';
/** 掉一个 OCR 结果该不该收、该不该提示重拍。 */
export function judgeOcr(result: OcrResult, policy: OcrQualityPolicy = DEFAULT_OCR_POLICY): OcrVerdict {
  if (result.error !== undefined || result.text.length === 0) {
    return 'reject';
  }
  if (result.confidence < policy.rejectThreshold) {
    return 'reject';
  }
  if (result.confidence < policy.retakeThreshold) {
    return 'retake';
  }
  return 'accept';
}
/* ========================================================================== */
/* 文件 7/12：projects/offlinelib-core/src/pipeline.ts */
/* 作用：库服务流水线编排 */
/* ========================================================================== */
/**
 * 流水线编排 —— 导入 → 分块 → 建索引 → 检索。
 *
 * THIS IS THE PART THE PPT CALLS "本项目的技术含量所在"
 *   开题 PPT 的 P9 写着：
 *     "① 本地索引与检索（切词 / 倒排 / BM25 / 高亮定位）——纯算法，可脱离设备做单元测试
 *      ② 端侧能力编排与调度——本项目的技术含量所在"
 *
 *   ① 在 index.ts / search.ts 里。② 就在这个文件。
 *
 *   编排的核心不是"把函数串起来"，而是**每一步都要做一次带信号的决策**：
 *
 *     感知环节  → 识别置信度 → 收下 / 提示重拍 / 丢弃       （信号：图像质量）
 *     索引环节  → 文档长度   → 分块大小 / 是否延迟建索引     （信号：内容规模）
 *     检索环节  → 得分分布   → 取几条                       （信号：得分断层）
 *     资源环节  → 延迟/内存/电量 → 还能取几条               （信号：资源）
 *
 *   **这四条用的是四个不同的信号。** 若把它们压成一个"剩余预算"，
```

```
 *   就退化成了我们实验里那个被每个固定策略严格支配的方案。
 *
 * □□ 纯 TS：不导入任何平台 Kit。OCR 由外部注入（见 ocr.ts）。
 */
import { InvertedIndex, chunkDocument, ChunkOptions, DEFAULT_CHUNK_OPTIONS } from './index';
import { search, decideBudget } from './search';
import { DocStore } from './store';
import { OcrEngine, judgeOcr, OcrVerdict, OcrQualityPolicy, DEFAULT_OCR_POLICY } from './ocr';
import {
  BudgetInput, Hit, SearchOptions, DocItem, DocSource, ScoreDiagnostics
} from './types';
/** 导入一份资料的输入。 */
export interface ImportInput {
  /** 用户可见的标题。 */
  title: string;
  /** 来源。决定是否需要 OCR。 */
  source: DocSource;
  /** 原始文件位置。 */
  uri: string;
  /**
   * 已经有文本时直接给（纯文本资料、或用户手工粘贴的笔记）。
   * 为空则走 OCR。
   */
  text?: string;
}
/** 导入的结果。 */
export interface ImportOutcome {
  /** 是否成功入库。 */
  ok: boolean;
  /** OCR 判定（纯文本导入时恒为 'accept'）。 */
  verdict: OcrVerdict;
  /** 入库后的文档 id（失败时为空串）。 */
  docId: string;
  /** 新建的块数。 */
  chunks: number;
  /** 给用户看的一句话（成功或失败原因）。 */
  message: string;
}
/** 一次查询的结果，含全部决策依据。 */
export interface QueryOutcome {
  hits: Hit[];
  k: number;
  kSource: 'fixed' | 'score-gap';
  diagnostics: ScoreDiagnostics;
  /** 资源预算给出的上限。 */
  budgetK: number;
  /** 资源预算的理由（"受时间预算限制"等）。 */
  budgetReason: string;
  /** 最终 k 是受信息信号还是资源信号限制。 */
  limitedBy: 'information' | 'resource';
```

```
}
/**
 * 资料库服务 —— 应用层唯一需要交互的入口。
 *
 * 把 store / index / ocr 三者组合起来，并保证：
 *   - 文档变更后索引与文档库保持一致
 *   - 每次检索都返回**决策依据**（便于界面向用户解释）
 */
export class LibraryService {
  private store: DocStore;
  private index: InvertedIndex;
  private ocr: OcrEngine;
  private chunkOpts: ChunkOptions;
  private ocrPolicy: OcrQualityPolicy;
  private seq: number;
  constructor(
    ocr: OcrEngine,
    chunkOpts: ChunkOptions = DEFAULT_CHUNK_OPTIONS,
    ocrPolicy: OcrQualityPolicy = DEFAULT_OCR_POLICY
  ) {
    this.store = new DocStore();
    this.index = new InvertedIndex();
    this.ocr = ocr;
    this.chunkOpts = chunkOpts;
    this.ocrPolicy = ocrPolicy;
    this.seq = 0;
  }
  /** 文档数。 */
  public docCount(): number {
    return this.store.size();
  }
  /** 索引块数。 */
  public chunkCount(): number {
    return this.index.chunkCount();
  }
  /** OCR 能力是否可用。**界面在提供"拍照导入"入口前必须先问这个。** */
  public ocrAvailable(): boolean {
    return this.ocr.available();
  }
  public ocrName(): string {
    return this.ocr.name();
  }
  /**
   * 导入一份资料。
   *
   * 决策点：OCR 置信度决定"收下 / 提示重拍 / 丢弃"。
   * 这是"感知环节由图像质量驱动"的具体实现。
   */
  public async importDocument(input: ImportInput): Promise<ImportOutcome> {
    let text = input.text !== undefined ? input.text : '';
```

```
let verdict: OcrVerdict = 'accept';
if (text.length === 0) {
  // 没有现成文本 → 走 OCR
  if (!this.ocr.available()) {
    return {
      ok: false,
      verdict: 'reject',
      docId: '',
      chunks: 0,
      message: '当前设备没有可用的 OCR 能力，请改用「粘贴文本」导入'
    };
  }
  const res = await this.ocr.recognize(input.uri);
  verdict = judgeOcr(res, this.ocrPolicy);
  if (verdict === 'reject') {
    return {
      ok: false,
      verdict: verdict,
      docId: '',
      chunks: 0,
      message: res.error !== undefined ? res.error : '识别质量过低，未入库'
    };
  }
  text = res.text;
}
if (text.length === 0) {
  return { ok: false, verdict: 'reject', docId: '', chunks: 0, message: '内容为空' };
}
this.seq += 1;
const docId = 'd' + this.seq.toString();
const doc: DocItem = {
  id: docId,
  title: input.title,
  source: input.source,
  uri: input.uri,
  addedAt: 0,
  text: text
};
this.store.put(doc);
// 决策点：内容规模决定分块大小
const opts = this.chunkOptionsFor(text.length);
const added = this.index.addDocument(doc, opts);
const base = verdict === 'retake'
  ? '已入库，但识别置信度偏低，建议重拍以获得更好的检索效果'
  : '已入库';
return {
  ok: true,
  verdict: verdict,
  docId: docId,
  chunks: added,
```

```
    message: base + '（' + added.toString() + ' 个片段）'
  };
}
/**
 * 由文档长度决定分块参数。
 *
 * 为什么不是固定值：短文档（一页笔记）按 240 字符切会切成一个块，
 * 而长文档（几十页 PDF）按 240 切会产生上千个块、索引膨胀。
 * **内容规模是一个独立信号**——它只影响分块，不该影响检索深度。
 */
private chunkOptionsFor(textLen: number): ChunkOptions {
  if (textLen <= 1000) {
    return { size: Math.max(120, textLen), overlap: 0 };
  }
  if (textLen <= 20000) {
    return { size: 240, overlap: 40 };
  }
  // 长文档：块更大以减少块数，保留重叠避免切断
  return { size: 400, overlap: 60 };
}
/**
 * 查询。
 *
 * **两个信号各管一件事**：
 *   - 得分分布 → 该取几条（信息层面）
 *   - 延迟/内存/电量 → 还取得起吗（资源层面）
 * 最终 k 取两者的较小值，并报告是谁在限制。
 */
public query(q: string, budget: BudgetInput, options: SearchOptions): QueryOutcome {
  const r = search(this.index, q, options);
  const bd = decideBudget(budget, this.averageChunkChars());
  let k = r.k;
  let limitedBy: 'information' | 'resource' = 'information';
  if (bd.k < r.k) {
    k = bd.k;
    limitedBy = 'resource';
  }
  const hits: Hit[] = r.hits.slice(0, k);
  return {
    hits: hits,
    k: k,
    kSource: r.kSource,
    diagnostics: r.diagnostics,
    budgetK: bd.k,
    budgetReason: bd.reason,
    limitedBy: limitedBy
  };
}
/**
 * 实际的块平均字符数。
```

```
 *
 * 早先这里写的是拿一个固定参数去推，那是错的——块大小随文档长度变，
 * 必须从**真实块**上量。这条注释留着，是因为"看起来合理的估计"
 * 正是这个项目里反复出错的形态。
 */
public averageChunkChars(): number {
  const n = this.index.chunkCount();
  if (n === 0) {
    return 0;
  }
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += this.index.getChunk(i).text.length;
  }
  return sum / n;
}
/** 序列化整个资料库（只存文档，索引在 load 时重建）。 */
public save(): string {
  return this.store.save();
}
/**
 * 恢复资料库并重建索引。
 *
 * 返回被跳过的坏条目数。**rebuildIndex 是必需的**——
 * 存档里只有文档，没有索引（见 store.ts 的设计决定）。
 */
public load(json: string): number {
  const skipped = this.store.load(json);
  this.seq = this.store.size();
  this.rebuildIndex();
  return skipped;
}
/** 从文档库重建索引。返回重建的块数。 */
public rebuildIndex(): number {
  this.index.clear();
  const docs = this.store.all();
  let n = 0;
  for (let i = 0; i < docs.length; i++) {
    const d = docs[i];
    n += this.index.addDocument(d, this.chunkOptionsFor(d.text.length));
  }
  return n;
}
/**
 * 删除一份资料。
 *
 * **不再全量重建索引**：早先这里写的是 `store.remove` + `rebuildIndex()`，
 * 删除一份文档要把全库重新切词、重新建倒排表。
 * 1000 份文档时删 10 份就是 10 次全量重建。
 *
```

```
 * 现在走 `InvertedIndex.removeDocument()` 的墓碑路径，
 * 代价只与该文档的块数成正比；墓碑累积到阈值时引擎会自动压缩。
 */
public removeDocument(id: string): boolean {
  const ok = this.store.remove(id);
  if (ok) {
    this.index.removeDocument(id);
  }
  return ok;
}
/**
 * 替换一份资料（用户重拍了照片、或重新粘贴了文本）。
 *
 * 比"先 removeDocument 再 importDocument"少一次全量重建，
 * 而且能保持文档在列表中的位置。
 */
public async replaceDocument(id: string, input: ImportInput): Promise<ImportOutcome> {
  const old = this.store.get(id);
  if (old === undefined) {
    return { ok: false, verdict: 'reject', docId: '', chunks: 0, message: '要替换的文档不存在' };
  }
  let text = input.text !== undefined ? input.text : '';
  let verdict: OcrVerdict = 'accept';
  if (text.length === 0) {
    if (!this.ocr.available()) {
      return {
        ok: false, verdict: 'reject', docId: '', chunks: 0,
        message: '当前设备没有可用的 OCR 能力，请改用「粘贴文本」导入'
      };
    }
    const res = await this.ocr.recognize(input.uri);
    verdict = judgeOcr(res, this.ocrPolicy);
    if (verdict === 'reject') {
      return {
        ok: false, verdict: verdict, docId: '', chunks: 0,
        message: res.error !== undefined ? res.error : '识别质量过低，未替换'
      };
    }
    text = res.text;
  }
  if (text.length === 0) {
    return { ok: false, verdict: 'reject', docId: '', chunks: 0, message: '内容为空' };
  }
  const doc: DocItem = {
    id: id,
    title: input.title,
    source: input.source,
    uri: input.uri,
    addedAt: old.addedAt,
    text: text
```

```
    };
    this.store.put(doc);
    const added = this.index.replaceDocument(doc, this.chunkOptionsFor(text.length));
    return {
      ok: true, verdict: verdict, docId: id, chunks: added,
      message: verdict === 'retake' ? '已替换，但识别置信度偏低，建议重拍' : '已替换'
    };
  }
  /** 墓碑数（死块）。界面可据此提示"索引需要整理"。 */
  public tombstoneCount(): number {
    return this.index.tombstoneCount();
  }
  /** 手动压缩索引。一般不需要——引擎会在墓碑超阈值时自动压缩。 */
  public compactIndex(): void {
    this.index.compact();
  }
  /** 训练/演示用：列出全部文档。 */
  public documents(): DocItem[] {
    return this.store.all();
  }
}
/* ========================================================================== */
/* 文件 8/12：app/offlinelib/entry/src/main/ets/lib/realocr.ets */
/* 作用：真实 OCR 实现（CoreVisionKit） */
/* ========================================================================== */
/**
 * 真实 OCR 引擎（CoreVisionKit）。
 *
 * □□ 这个文件**静态导入了 HMS 的 CoreVisionKit**。
 *
 *   为什么这是安全的：我们先用 `canIUse('SystemCapability.AI.OCR.TextRecognition')`
 *   在设备上实测过，返回 **true**（截图见 deliverables/演示素材/L3真机验证/）。
 *
 *   为什么仍然要谨慎：早期版本静态导入 `@hms.ai.A2A` 时，因为设备缺少对应 HSP，
 *   应用在**模块加载期**就 abort 了。同一个 deviceType(2in1) 的 AgentKit
 *   探针现在仍返回 false —— 说明这两个 Kit 在这台设备上的可用性并不一样。
 *
 *   所以：**这个文件只应该在 canIUse 为 true 的设备上被加载。**
 *   实际使用中由 pipeline 决定注入 Real 还是 Fake 引擎。
 *
 * 它实现的是 lib/ocr.ts 里的 `OcrEngine` 接口 —— 核心算法层完全不知道
 * 这里是 Vision Kit 还是假引擎，这正是当初把 OCR 做成接口的目的。
 */
import { textRecognition } from '@kit.CoreVisionKit';
import { image } from '@kit.ImageKit';
import { common } from '@kit.AbilityKit';
import { OcrEngine, OcrResult } from './ocr';
export class RealOcrEngine implements OcrEngine {
  private ctx: common.UIAbilityContext;
  constructor(ctx: common.UIAbilityContext) {
```

```
  this.ctx = ctx;
}
public name(): string {
  return 'vision-kit-ocr';
}
/**
 * 能力是否可用。
 *
 * 用 `canIUse` 而不是"试着调一下"：前者不会产生副作用，也不会因为失败而拖慢启动。
 */
public available(): boolean {
  return canIUse('SystemCapability.AI.OCR.TextRecognition');
}
/**
 * 识别一张图。
 *
 * `imageUri` 支持两种形式：
 *   · `rawfile:<文件名>` —— 应用内置资源（用于自测，不依赖相机）
 *   · 其他               —— 交给上层扩展（相机/相册路径）
 */
public async recognize(imageUri: string): Promise<OcrResult> {
  const t0: number = Date.now();
  try {
    let buf: ArrayBuffer | undefined = undefined;
    if (imageUri.startsWith('rawfile:')) {
      const name: string = imageUri.substring('rawfile:'.length);
      const content: Uint8Array = await this.ctx.resourceManager.getRawFileContent(name);
      // Uint8Array 的 buffer 可能带偏移，这里拷一份干净的 ArrayBuffer
      const copy: Uint8Array = new Uint8Array(content.length);
      copy.set(content);
      buf = copy.buffer as ArrayBuffer;
    } else {
      // 相机/相册路径尚未接入 —— 明确报错而不是静默失败
      return {
        text: '',
        confidence: 0,
        elapsedMs: Date.now() - t0,
        error: '尚未接入相机/相册图片路径，当前只支持 rawfile: 内置图'
      };
    }
    const src: image.ImageSource = image.createImageSource(buf);
    const pm: image.PixelMap = await src.createPixelMap();
    await src.release();
    const conf: textRecognition.TextRecognitionConfiguration = {
      isDirectionDetectionSupported: false
    };
    const res: textRecognition.TextRecognitionResult = await textRecognition.recognizeText(
      { pixelMap: pm }, conf
    );
    await pm.release();
```

```
      const text: string = res.value;
      // Vision Kit 不返回标量置信度，只返回分块。
      // 这里用"块数"做一个保守的代理指标：块越少越可能是极短或识别失败。
      // **不要把这个数当成概率**——它只是给 OcrQualityPolicy 一个有界的输入。
      const blockCount: number = res.blocks !== undefined ? res.blocks.length : 0;
      const confidence: number = text.length === 0 ? 0 : (blockCount >= 3 ? 0.9 : 0.7);
      return { text: text, confidence: confidence, elapsedMs: Date.now() - t0 };
    } catch (e) {
      const msg: string = e instanceof Error ? e.message : '未知错误';
      return {
        text: '',
        confidence: 0,
        elapsedMs: Date.now() - t0,
        error: 'OCR 失败：' + msg
      };
    }
  }
}
/* ========================================================================== */
/* 文件 9/12：app/offlinelib/entry/src/main/ets/pages/Index.ets */
/* 作用：主界面 */
/* ========================================================================== */
/**
 * 离线资料库 —— 主界面。
 *
 * 设计原则（都来自开题答辩时老师提的批评）：
 *   1. **打开就能用**：默认进搜索页，不需要先"建库"才能看到东西
 *   2. **可解释**：每次检索都告诉用户"为什么给你这几条"（k 是怎么定的）
 *   3. **诚实降级**：OCR 不可用时明确提示改用粘贴文本，而不是静默失败
 *
 * □□ ArkTS 约束（L2 验证时实测踩到的，务必遵守）：
 *   · 对象字面量必须有上下文类型
 *   · 数组字面量元素类型必须可推断
 *   · 禁止 any / unknown / as unknown as
 */
import { LibraryService } from '../lib/pipeline';
import { FakeOcrEngine } from '../lib/ocr';
import { DocItem, Hit, Span, BudgetInput, SearchOptions, DocSource } from '../lib/types';
import { QueryOutcome, ImportOutcome } from '../lib/pipeline';
import { runSelfChecks, CheckSummary, CheckResult } from '../selfcheck/SelfCheck';
import { probeCapabilities, CapabilityProbe } from '../selfcheck/Capability';
import { common } from '@kit.AbilityKit';
import { RealOcrEngine } from '../lib/realocr';
import { OcrResult } from '../lib/ocr';
const BUDGET: BudgetInput = { latencyBudgetMs: 500, memoryBudgetKb: 4096, batteryPercent: 80 };
const OPTS_FIXED: SearchOptions = { topK: 3, minK: 1, maxK: 20 };
const OPTS_AUTO: SearchOptions = { topK: 0, minK: 1, maxK: 20 };
@Entry
@Component
struct Index {
```

```
// 服务实例（用假 OCR：本页要在没有 HMS 的环境里也能跑）
private svc: LibraryService = new LibraryService(new FakeOcrEngine());
@State currentTab: number = 0;
@State query: string = '';
@State hits: Hit[] = [];
@State kUsed: number = 0;
@State kSource: string = '-';
@State limitedBy: string = '-';
@State budgetReason: string = '-';
@State candidateCount: number = 0;
@State searchStatus: string = '输入关键词，或先在「资料」页添加一份内容';
@State useAutoK: boolean = true;
@State docs: DocItem[] = [];
@State docCount: number = 0;
@State chunkCount: number = 0;
@State tombstone: number = 0;
// 导入表单
@State inputTitle: string = '';
@State inputText: string = '';
@State importStatus: string = '';
// 自测
@State checks: CheckResult[] = [];
@State checkSummary: string = '尚未运行';
@State checksDone: boolean = false;
@State caps: CapabilityProbe[] = [];
@State ocrStatus: string = '未运行';
@State ocrText: string = '';
aboutToAppear(): void {
  this.caps = probeCapabilities();
  this.refreshStats();
  this.seedDemo();
}
// ---------------------------------------------------------------------
// 数据操作
// ---------------------------------------------------------------------
/** 首次启动放一份示例内容，避免"打开是空的、不知道能干嘛"。 */
private seedDemo(): void {
  if (this.svc.docCount() > 0) {
    return;
  }
  const demo: string = '示例资料：移动端的多智能体系统存在预算矛盾，' +
    '普通对话消耗一倍 token，而多智能体约为十五倍；手机能提供的上下文窗口只有四千到八千。' +
    '因此取舍是强制的：注入哪些工具、消息用什么形式、这一步在哪里执行，都要在预算内决定。';
  this.svc.importDocument({ title: '示例：预算调度笔记', source: 'note', uri: 'demo://1', text: demo })
    .then((_r: ImportOutcome) => { this.refreshStats(); });
}
private refreshStats(): void {
  this.docs = this.svc.documents();
  this.docCount = this.svc.docCount();
  this.chunkCount = this.svc.chunkCount();
```

```
  this.tombstone = this.svc.tombstoneCount();
}
private doSearch(): void {
  const q: string = this.query.trim();
  if (q.length === 0) {
    this.searchStatus = '请先输入关键词';
    return;
  }
  if (this.svc.chunkCount() === 0) {
    this.searchStatus = '资料库还是空的，请先在「资料」页添加内容';
    return;
  }
  const opts: SearchOptions = this.useAutoK ? OPTS_AUTO : OPTS_FIXED;
  const out: QueryOutcome = this.svc.query(q, BUDGET, opts);
  this.hits = out.hits;
  this.kUsed = out.k;
  this.kSource = out.kSource;
  this.limitedBy = out.limitedBy;
  this.budgetReason = out.budgetReason;
  this.candidateCount = out.diagnostics.candidateCount;
  this.searchStatus = out.hits.length === 0
    ? '没有找到相关内容'
    : `找到 ${out.hits.length} 条（候选 ${out.diagnostics.candidateCount} 条）`;
}
private doImport(): void {
  const t: string = this.inputText.trim();
  if (t.length === 0) {
    this.importStatus = '内容不能为空';
    return;
  }
  const title: string = this.inputTitle.trim().length > 0 ? this.inputTitle.trim() : '未命名资料';
  const src: DocSource = 'note';
  this.svc.importDocument({ title: title, source: src, uri: 'note://' + Date.now().toString(), text: t
      })
    .then((r: ImportOutcome) => {
      this.importStatus = r.message;
      if (r.ok) {
        this.inputTitle = '';
        this.inputText = '';
        this.refreshStats();
      }
    })
    .catch((_e: Error) => { this.importStatus = '导入失败'; });
}
private doRemove(id: string): void {
  this.svc.removeDocument(id);
  this.refreshStats();
  this.hits = [];
  this.searchStatus = '已删除，请重新搜索';
}
```

```
private runChecks(): void {
  this.checkSummary = '运行中…';
  runSelfChecks()
    .then((s: CheckSummary) => {
      this.checks = s.results;
      this.checksDone = true;
      this.checkSummary = s.failed === 0
        ? `全部通过：${s.passed} / ${s.total}`
        : `有失败：${s.failed} / ${s.total}`;
    })
    .catch((_e: Error) => { this.checkSummary = '自测运行失败'; });
}
/** L4：真实 OCR 实测（内置测试图，不依赖相机）。 */
private runOcrTest(): void {
  this.ocrStatus = '识别中…';
  this.ocrText = '';
  try {
    // getContext(this) 在页面里拿到的就是 UIAbilityContext
    const ctx = getContext(this) as common.UIAbilityContext;
    const engine = new RealOcrEngine(ctx);
    if (!engine.available()) {
      this.ocrStatus = '该设备不支持 OCR';
      return;
    }
    engine.recognize('rawfile:ocr_sample.png')
      .then((r: OcrResult) => {
        if (r.error !== undefined) {
          this.ocrStatus = r.error;
          return;
        }
        this.ocrStatus = `识别成功：${r.text.length} 字 · 用时 ${r.elapsedMs} ms`;
        this.ocrText = r.text;
      })
      .catch((e: Error) => { this.ocrStatus = 'OCR 调用失败：' + e.message; });
  } catch (e) {
    this.ocrStatus = 'OCR 初始化失败';
  }
}
// ---------------------------------------------------------------------------
// 视图片段
// ---------------------------------------------------------------------------
/** 把带高亮的文本渲染成多个 Span。 */
@Builder
highlightText(text: string, spans: Span[]) {
  Text() {
    Span(text.substring(0, spans.length > 0 ? spans[0].start : text.length))
    ForEach(spans, (s: Span) => {
      Span(text.substring(s.start, s.end))
        .fontColor('#C0392B')
        .fontWeight(FontWeight.Bold)
```

```
    }, (_s: Span, i: number) => i.toString())
    if (spans.length > 0) {
      Span(text.substring(spans[spans.length - 1].end))
    }
  }
  .fontSize(13)
  .width('100%')
}
@Builder
searchTab() {
  Column({ space: 10 }) {
    Row({ space: 8 }) {
      TextInput({ placeholder: '搜索你的资料…', text: this.query })
        .layoutWeight(1)
        .height(42)
        .fontSize(14)
        .onChange((v: string) => { this.query = v; })
        .onSubmit(() => { this.doSearch(); })
      Button('搜索')
        .height(42)
        .fontSize(14)
        .onClick(() => { this.doSearch(); })
    }
    .width('100%')
    Row({ space: 8 }) {
      Text('检索深度').fontSize(12).fontColor('#666666')
      Toggle({ type: ToggleType.Switch, isOn: this.useAutoK })
        .onChange((on: boolean) => { this.useAutoK = on; })
      Text(this.useAutoK ? '按得分分布自适应' : '固定 3 条')
        .fontSize(12)
        .fontColor('#0A59F7')
    }
    .width('100%')
    // 决策解释条
    if (this.kUsed > 0) {
      Column({ space: 3 }) {
        Text(`返回 ${this.kUsed} 条 · 来源 ${this.kSource} · 限制方 ${this.limitedBy}`)
          .fontSize(11).fontColor('#0A59F7').width('100%')
        Text(`候选 ${this.candidateCount} 条 · 资源约束：${this.budgetReason}`)
          .fontSize(10).fontColor('#888888').width('100%')
      }
      .padding(8)
      .width('100%')
      .backgroundColor('#EEF4FF')
      .borderRadius(6)
    }
    Text(this.searchStatus).fontSize(11).fontColor('#888888').width('100%')
    List({ space: 8 }) {
      ForEach(this.hits, (h: Hit) => {
        ListItem() {
```

```
          Column({ space: 4 }) {
            Row({ space: 6 }) {
              Text(this.titleOf(h.chunk.docId)).fontSize(13).fontWeight(FontWeight.Medium).layoutWeight
                  (1)
              Text(h.score.toFixed(2)).fontSize(10).fontColor('#0A59F7')
            }
            .width('100%')
            this.highlightText(h.chunk.text, h.spans)
          }
          .padding(10)
          .width('100%')
          .backgroundColor(Color.White)
          .borderRadius(8)
        }
      }, (h: Hit) => h.chunk.id)
    }
    .layoutWeight(1)
    .width('100%')
  }
  .padding(14)
  .width('100%')
  .height('100%')
  .backgroundColor('#F5F7FA')
}
private titleOf(docId: string): string {
  for (let i = 0; i < this.docs.length; i++) {
    if (this.docs[i].id === docId) {
      return this.docs[i].title;
    }
  }
  return docId;
}
@Builder
libraryTab() {
  Column({ space: 10 }) {
    Row({ space: 12 }) {
      Text(`资料 ${this.docCount} 份`).fontSize(12).fontColor('#333333')
      Text(`片段 ${this.chunkCount} 个`).fontSize(12).fontColor('#333333')
      if (this.tombstone > 0) {
        Text(`待整理 ${this.tombstone}`).fontSize(12).fontColor('#C0392B')
      }
    }
    .width('100%')
    Column({ space: 8 }) {
      TextInput({ placeholder: '标题（可留空）', text: this.inputTitle })
        .height(40).fontSize(13)
        .onChange((v: string) => { this.inputTitle = v; })
      TextArea({ placeholder: '粘贴或输入要入库的内容…', text: this.inputText })
        .height(110).fontSize(13)
        .onChange((v: string) => { this.inputText = v; })
```

```
      Row({ space: 8 }) {
        Button('加入资料库')
          .height(38).fontSize(13)
          .onClick(() => { this.doImport(); })
        if (this.importStatus.length > 0) {
          Text(this.importStatus).fontSize(11).fontColor('#0A8F4C').layoutWeight(1)
        }
      }
      .width('100%')
    }
    .padding(10)
    .width('100%')
    .backgroundColor(Color.White)
    .borderRadius(8)
    List({ space: 6 }) {
      ForEach(this.docs, (d: DocItem) => {
        ListItem() {
          Row({ space: 8 }) {
            Column({ space: 2 }) {
              Text(d.title).fontSize(13).width('100%')
              Text(`${d.source} · ${d.text.length} 字`).fontSize(10).fontColor('#888888').width('100%')
            }
            .layoutWeight(1)
            .alignItems(HorizontalAlign.Start)
            Button('删除')
              .fontSize(11).height(28)
              .backgroundColor('#F0F0F0').fontColor('#C0392B')
              .onClick(() => { this.doRemove(d.id); })
          }
          .padding(10)
          .width('100%')
          .backgroundColor(Color.White)
          .borderRadius(8)
        }
      }, (d: DocItem) => d.id)
    }
    .layoutWeight(1)
    .width('100%')
  }
  .padding(14)
  .width('100%')
  .height('100%')
  .backgroundColor('#F5F7FA')
}
@Builder
advancedTab() {
  Column({ space: 10 }) {
    // 能力探测（canIUse，零风险：不 import 任何 Kit）
    Column({ space: 5 }) {
      Text('设备能力探测（canIUse）').fontSize(14).fontWeight(FontWeight.Bold).width('100%')
```

```
  Text('只用 canIUse 查询，不 import 任何 Kit —— 因此不可能把应用拖崩')
    .fontSize(10).fontColor('#888888').width('100%')
  ForEach(this.caps, (c: CapabilityProbe) => {
    Row({ space: 8 }) {
      Text(c.available ? '□' : '□')
        .fontSize(13).width(16)
        .fontColor(c.available ? '#0A8F4C' : '#C0392B')
      Column({ space: 1 }) {
        Text(c.label).fontSize(11.5).width('100%')
        Text(c.syscap.replace('SystemCapability.', '')).fontSize(9)
          .fontColor('#999999').width('100%')
      }
      .layoutWeight(1)
      .alignItems(HorizontalAlign.Start)
    }
    .width('100%')
    .padding({ left: 8, right: 8, top: 4, bottom: 4 })
  }, (c: CapabilityProbe) => c.syscap)
}
.padding(12)
.width('100%')
.backgroundColor(Color.White)
.borderRadius(8)
// L4：真实 OCR 实测
Column({ space: 6 }) {
  Text('OCR 实测（CoreVisionKit）').fontSize(14).fontWeight(FontWeight.Bold).width('100%')
  Text('用内置测试图跑一次真实 OCR —— 不依赖相机')
    .fontSize(10).fontColor('#888888').width('100%')
  Row({ space: 10 }) {
    Button('运行 OCR 实测')
      .height(36).fontSize(13)
      .onClick(() => { this.runOcrTest(); })
    Text(this.ocrStatus).fontSize(11).fontColor('#666666').layoutWeight(1)
  }
  .width('100%')
  if (this.ocrText.length > 0) {
    Text(this.ocrText)
      .fontSize(11)
      .width('100%')
      .padding(8)
      .backgroundColor('#F5F7FA')
      .borderRadius(6)
  }
}
.padding(12)
.width('100%')
.backgroundColor(Color.White)
.borderRadius(8)
Column({ space: 6 }) {
  Text('四个环节，四个信号').fontSize(14).fontWeight(FontWeight.Bold).width('100%')
```

```
  Text('这是本项目的研究主张：调度不能用单一的「剩余预算」驱动。')
    .fontSize(11).fontColor('#666666').width('100%')
  Text('· 感知（OCR）→ 识别置信度决定收下/重拍/丢弃')
    .fontSize(11).width('100%')
  Text('· 索引 → 内容规模决定分块大小')
    .fontSize(11).width('100%')
  Text('· 检索 → 得分分布决定返回几条')
    .fontSize(11).width('100%')
  Text('· 资源 → 延迟/内存/电量决定还取得起几条')
    .fontSize(11).width('100%')
}
.padding(12)
.width('100%')
.backgroundColor(Color.White)
.borderRadius(8)
Column({ space: 8 }) {
  Row({ space: 8 }) {
    Button('运行设备内自测')
      .height(38).fontSize(13)
      .onClick(() => { this.runChecks(); })
    Text(this.checkSummary).fontSize(12)
      .fontColor(this.checkSummary.indexOf('全部通过') >= 0 ? '#0A8F4C' : '#666666')
      .layoutWeight(1)
  }
  .width('100%')
  if (this.checksDone) {
    List({ space: 1 }) {
      ForEach(this.checks, (c: CheckResult) => {
        ListItem() {
          Row({ space: 8 }) {
            Text(c.passed ? '□' : '□')
              .fontSize(13).width(18)
              .fontColor(c.passed ? '#0A8F4C' : '#C0392B')
            Text(c.name).fontSize(11).layoutWeight(1)
            if (!c.passed) {
              Text(c.detail).fontSize(10).fontColor('#C0392B')
            }
          }
          .padding({ left: 8, right: 8, top: 5, bottom: 5 })
          .width('100%')
        }
      }, (c: CheckResult, i: number) => c.name + i.toString())
    }
    .layoutWeight(1)
    .width('100%')
  }
}
.padding(12)
.width('100%')
.layoutWeight(1)
```

```
      .backgroundColor(Color.White)
      .borderRadius(8)
    }
    .padding(14)
    .width('100%')
    .height('100%')
    .backgroundColor('#F5F7FA')
  }
  build() {
    Column() {
      Tabs({ barPosition: BarPosition.End, index: this.currentTab }) {
        TabContent() { this.searchTab() }.tabBar('搜索')
        TabContent() { this.libraryTab() }.tabBar('资料')
        TabContent() { this.advancedTab() }.tabBar('高级')
      }
      .onChange((i: number) => { this.currentTab = i; })
      .barHeight(52)
      .width('100%')
      .height('100%')
    }
    .width('100%')
    .height('100%')
    .backgroundColor('#F5F7FA')
  }
}
/* ========================================================================== */
/* 文件 10/12：app/offlinelib/entry/src/main/ets/selfcheck/Capability.ets */
/* 作用：设备能力探测 */
/* ========================================================================== */
/**
 * 设备能力探测 —— 用 `canIUse` 查系统能力是否存在。
 *
 * WHY canIUse AND NOT A DIRECT IMPORT
 *   这个工程在早期版本里踩过一个真实的坑：
 *   应用静态 `import { createA2AServer } from '@hms.ai.A2A'`，
 *   而设备上没有对应的 HSP 时，ArkTS 运行时会在**模块加载期**直接 abort 整个进程——
 *   不是"功能不可用"，是"应用起不来"（崩溃日志里是 LoadJSPandaFile ... abort）。
 *
 *   所以探测能力必须用 `canIUse`：它是全局函数，**不需要 import 任何模块**，
 *   因此不存在把应用拖死的风险。
 *
 *   `canIUse` 返回 true 只说明"这个 syscap 被声明可用"，
 *   不代表 API 一定调得通；但返回 false 就**一定别去 import**。
 */
/** 一条能力探测结果。 */
export interface CapabilityProbe {
  /** 界面上显示的名字。 */
  label: string;
  /** syscap 全名。 */
  syscap: string;
```

```
  /** canIUse 的结果。 */
  available: boolean;
  /** 这个能力在本项目里用来做什么。 */
  purpose: string;
}
/**
 * 探测本项目关心的四项能力。
 *
 * 顺序刻意从"最需要"到"最不需要"：
 * OCR 是资料库入口，检索是核心，VisionBase 是 OCR 的底座，
 * AgentKit 只是拿来和当年崩溃的 A2A 做对照。
 */
export function probeCapabilities(): CapabilityProbe[] {
  const out: CapabilityProbe[] = [];
  const ocr: CapabilityProbe = {
    label: 'OCR 文字识别',
    syscap: 'SystemCapability.AI.OCR.TextRecognition',
    available: canIUse('SystemCapability.AI.OCR.TextRecognition'),
    purpose: '拍照导入：把图片转成可检索的文字'
  };
  out.push(ocr);
  const retrieval: CapabilityProbe = {
    label: '端侧检索',
    syscap: 'SystemCapability.DataAugmentation.Retrieval',
    available: canIUse('SystemCapability.DataAugmentation.Retrieval'),
    purpose: '系统级向量召回（本项目目前用自建 BM25，不依赖它）'
  };
  out.push(retrieval);
  const vision: CapabilityProbe = {
    label: '视觉基础能力',
    syscap: 'SystemCapability.AI.Vision.VisionBase',
    available: canIUse('SystemCapability.AI.Vision.VisionBase'),
    purpose: 'OCR 的底座，通常与 OCR 同时存在'
  };
  out.push(vision);
  const agent: CapabilityProbe = {
    label: 'AgentKit',
    syscap: 'SystemCapability.AI.Agent.AgentKit',
    available: canIUse('SystemCapability.AI.Agent.AgentKit'),
    purpose: '对照组：当年静态导入 A2A 导致加载期崩溃的那一族'
  };
  out.push(agent);
  return out;
}
/* ========================================================================== */
/* 文件 11/12：app/offlinelib/entry/src/main/ets/selfcheck/SelfCheck.ets */
/* 作用：设备内自测 */
/* ========================================================================== */
/**
 * 设备内自测 —— 把关键断言搬到应用里跑，结果直接显示在屏幕上。
```

```
 *
 * WHY A SELF-TEST PAGE INSTEAD OF JUST NODE TESTS
 *   Node 单测证明了算法在**电脑上**是对的，但这不能证明两件事：
 *     1. 这些 .ts 能被 ArkTS 编译（语法/类型子集是否真的兼容）
 *     2. 在设备的 ArkTS 运行时里行为是否一致
 *
 *   所以这一页的作用是：**打开应用就能看到关键用例是否通过**。
 *   它是可以截图当证据的——这直接对应老师那条"技术路线无法核实"。
 *
 * □□ 这一页刻意**不依赖任何 HMS Kit**（只用假 OCR 引擎、不碰网络）。
 *    原因见 lib/ocr.ts：缺少 HSP 时静态导入会让应用在加载期崩溃。
 *
 * □□ ArkTS 的几条硬约束（这次实测踩到的）：
 *   · 对象字面量必须**有上下文类型**（arkts-no-untyped-obj-literals）
 *   · 数组字面量的元素类型必须可推断（arkts-no-noninferrable-arr-literals）
 *   · 禁止 any / unknown（arkts-no-any-unknown）
 *   所以下面每一步都显式标注类型，不用 `as`、不用 `as unknown as`。
 */
import { InvertedIndex, chunkDocument, locateSpans, Candidate } from '../lib/index';
import { search, decideKByScoreGap, decideBudget } from '../lib/search';
import { DocStore } from '../lib/store';
import { FakeOcrEngine, judgeOcr, OcrResult, OcrVerdict } from '../lib/ocr';
import { LibraryService } from '../lib/pipeline';
import { DocItem, Chunk, SearchOptions, BudgetInput, BudgetDecision } from '../lib/types';
/** 一条自测结果。 */
export interface CheckResult {
  name: string;
  passed: boolean;
  detail: string;
}
/** 自测汇总。 */
export interface CheckSummary {
  total: number;
  passed: number;
  failed: number;
  results: CheckResult[];
}
const BUDGET: BudgetInput = { latencyBudgetMs: 100000, memoryBudgetKb: 1000000, batteryPercent: 90 };
const OPTS_AUTO: SearchOptions = { topK: 0, minK: 1, maxK: 10 };
function makeDoc(id: string, text: string): DocItem {
  const d: DocItem = { id: id, title: 't-' + id, source: 'note', uri: 'mem://' + id, addedAt: 1, text: te
    xt };
  return d;
}
function makeChunk(id: string, text: string): Chunk {
  const c: Chunk = { id: id, docId: 'd', seq: 0, start: 0, end: text.length, text: text };
  return c;
}
function makeOcr(text: string, confidence: number): OcrResult {
  const r: OcrResult = { text: text, confidence: confidence, elapsedMs: 1 };
```

```
  return r;
}
/**
 * 跑全部自测。
 *
 * 返回结构化结果而不是抛异常：一页里有一个失败，不该让整页白屏。
 */
export async function runSelfChecks(): Promise<CheckSummary> {
  const results: CheckResult[] = [];
  const record = (name: string, ok: boolean, detail: string): void => {
    const r: CheckResult = { name: name, passed: ok, detail: detail };
    results.push(r);
  };
  const check = (name: string, fn: () => void): void => {
    let ok: boolean = true;
    let detail: string = 'ok';
    try {
      fn();
    } catch (err) {
      ok = false;
      // 不读 err.message：ArkTS 里 catch 参数类型是受限的，
      // 而且断言失败的原因对我们够用了——失败的用例名已经说明了问题。
      detail = '断言未通过';
    }
    record(name, ok, detail);
  };
  const assertTrue = (cond: boolean, msg: string): void => {
    if (!cond) {
      throw new Error(msg);
    }
  };
  const assertStr = (a: string, b: string, msg: string): void => {
    if (a !== b) {
      throw new Error(msg + '：期望 ' + b + '，实际 ' + a);
    }
  };
  const assertNum = (a: number, b: number, msg: string): void => {
    if (a !== b) {
      throw new Error(msg + '：期望 ' + b.toString() + '，实际 ' + a.toString());
    }
  };
  // ---- 纯算法 ----
  check('索引：加入块后尺寸正确', () => {
    const idx = new InvertedIndex();
    idx.addChunk(makeChunk('c', '离线资料库'));
    assertNum(idx.size(), 1, '应有 1 个活块');
  });
  check('切词：英文小写化后可检索', () => {
    const idx = new InvertedIndex();
    idx.addDocument(makeDoc('e', 'BM25 Ranking is statistical'));
```

```
  assertNum(idx.score('ranking').length, 1, '小写查询应命中大写原文');
});
check('分块：长文本切成多块且覆盖到文末', () => {
  const text: string = 'x'.repeat(1000);
  const d: DocItem = makeDoc('l', text);
  const cs: Chunk[] = chunkDocument(d, { size: 240, overlap: 40 });
  assertTrue(cs.length > 1, '应切成多块');
  assertNum(cs[cs.length - 1].end, 1000, '末块应到达文末');
});
check('BM25：含查询词的资料排第一', () => {
  const idx = new InvertedIndex();
  idx.addDocument(makeDoc('a', '这里讨论预算调度与上下文窗口'));
  idx.addDocument(makeDoc('b', '检索置信度决定注入深度'));
  const cands: Candidate[] = idx.score('检索置信度');
  assertTrue(cands.length > 0, '应有候选');
  assertStr(cands[0].chunk.docId, 'b', '含查询词的资料应排第一');
});
check('高亮：定位到原文区间', () => {
  const matched: Map<string, number> = new Map<string, number>();
  matched.set('检索', 2);
  const spans = locateSpans('检索与检索', matched);
  assertTrue(spans.length >= 1, '应产生高亮区间');
  assertNum(spans[0].start, 0, '首个区间应从 0 开始');
});
check('自适应 k：明显断层处截断', () => {
  const c0: Candidate = { index: 0, chunk: makeChunk('a', 'x'), score: 10, matched: new Map<string, num
    ber>() };
  const c1: Candidate = { index: 1, chunk: makeChunk('b', 'y'), score: 9, matched: new Map<string, numb
    er>() };
  const c2: Candidate = { index: 2, chunk: makeChunk('c', 'z'), score: 1, matched: new Map<string, numb
    er>() };
  const cands: Candidate[] = [c0, c1, c2];
  const k: number = decideKByScoreGap(cands, 1, 10);
  assertNum(k, 2, '应保留断层之前的 2 条');
});
check('预算决策：极小预算也至少 1 条', () => {
  const tight: BudgetInput = { latencyBudgetMs: 0, memoryBudgetKb: 0, batteryPercent: 5 };
  const d: BudgetDecision = decideBudget(tight, 240);
  assertNum(d.k, 1, 'k 不应小于 1');
});
// ---- 存储与 OCR ----
check('存储：JSON 往返保持内容', () => {
  const s: DocStore = new DocStore();
  s.put(makeDoc('a', '第一份'));
  s.put(makeDoc('b', '第二份'));
  const s2: DocStore = new DocStore();
  const skipped: number = s2.load(s.save());
  assertNum(skipped, 0, '不应跳过任何条目');
  assertNum(s2.size(), 2, '应恢复 2 份文档');
});
```

```
check('存储：坏数据被跳过而非整体失败', () => {
  const payload: string = '{"version":1,"docs":[' +
    '{"id":"","title":"t","source":"note","uri":"u","addedAt":1,"text":"x"},' +
    '{"id":"good","title":"t","source":"note","uri":"u","addedAt":1,"text":"y"}]}';
  const s: DocStore = new DocStore();
  const skipped: number = s.load(payload);
  assertNum(skipped, 1, '应跳过 1 条坏数据');
  assertNum(s.size(), 1, '应保留 1 条好数据');
});
check('OCR 判定：高中低三档', () => {
  const hi: OcrVerdict = judgeOcr(makeOcr('清晰', 0.95));
  const mid: OcrVerdict = judgeOcr(makeOcr('一般', 0.4));
  const lo: OcrVerdict = judgeOcr(makeOcr('很糊', 0.1));
  assertStr(hi, 'accept', '高置信度应收下');
  assertStr(mid, 'retake', '中置信度应提示重拍');
  assertStr(lo, 'reject', '低置信度应丢弃');
});
// ---- 流水线（异步） ----
const svc: LibraryService = new LibraryService(new FakeOcrEngine());
let firstId: string = '';
{
  let ok: boolean = true;
  let detail: string = 'ok';
  let chunks: number = 0;
  try {
    const r = await svc.importDocument({
      title: '预算笔记', source: 'note', uri: 'u1',
      text: '移动端的多智能体系统存在预算矛盾，上下文窗口只有四千到八千。'
    });
    ok = r.ok;
    chunks = r.chunks;
    firstId = r.docId;
    if (!ok) {
      detail = r.message;
    }
  } catch (err) {
    ok = false;
    detail = '导入抛异常';
  }
  record('流水线：纯文本导入成功', ok && chunks > 0, ok ? 'ok' : detail);
}
{
  let ok: boolean = true;
  let detail: string = 'ok';
  try {
    const q = svc.query('预算矛盾', BUDGET, OPTS_AUTO);
    ok = q.hits.length >= 1 && q.hits[0].spans.length > 0;
    if (!ok) {
      detail = '命中 ' + q.hits.length.toString() + ' 条';
    }
```

```
  } catch (err) {
    ok = false;
    detail = '查询抛异常';
  }
  record('流水线：能检索到刚导入的内容（带高亮）', ok, detail);
}
{
  let ok: boolean = true;
  let detail: string = 'ok';
  try {
    const r = await svc.importDocument({
      title: '另一份', source: 'note', uri: 'u2', text: '完全不同的内容甲乙丙丁'
    });
    const before: number = svc.docCount();
    svc.removeDocument(r.docId);
    const after: number = svc.docCount();
    const hitsAfter: number = svc.query('甲乙丙丁', BUDGET, OPTS_AUTO).hits.length;
    ok = before === 2 && after === 1 && hitsAfter === 0;
    if (!ok) {
      detail = '删除前 ' + before.toString() + ' / 删除后 ' + after.toString() +
        ' / 残留命中 ' + hitsAfter.toString();
    }
  } catch (err) {
    ok = false;
    detail = '删除抛异常';
  }
  record('流水线：增量删除后检索不到', ok, detail);
}
{
  let ok: boolean = true;
  let detail: string = 'ok';
  try {
    const json: string = svc.save();
    const svc2: LibraryService = new LibraryService(new FakeOcrEngine());
    svc2.load(json);
    const q2 = svc2.query('预算矛盾', BUDGET, OPTS_AUTO);
    ok = q2.hits.length >= 1;
    if (!ok) {
      detail = '恢复后命中 0 条（索引未重建？）';
    }
  } catch (err) {
    ok = false;
    detail = '存档往返抛异常';
  }
  record('流水线：存档恢复后仍能检索（索引重建）', ok, detail);
}
let passed: number = 0;
for (let i = 0; i < results.length; i++) {
  if (results[i].passed) {
    passed += 1;
```

```
    }
  }
  const summary: CheckSummary = {
    total: results.length,
    passed: passed,
    failed: results.length - passed,
    results: results
  };
  return summary;
}
/* ========================================================================== */
/* 文件 12/12：app/offlinelib/entry/src/main/ets/entryability/EntryAbility.ets */
/* 作用：应用入口 */
/* ========================================================================== */
import { AbilityConstant, UIAbility, Want } from '@kit.AbilityKit';
import { hilog } from '@kit.PerformanceAnalysisKit';
import { window } from '@kit.ArkUI';
export default class EntryAbility extends UIAbility {
  onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onCreate');
  }
  onDestroy(): void {
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onDestroy');
  }
  onWindowStageCreate(windowStage: window.WindowStage): void {
    // Main window is created, set main page for this ability
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onWindowStageCreate');
    windowStage.loadContent('pages/Index', (err) => {
      if (err.code) {
        hilog.error(0x0000, 'testTag', 'Failed to load the content. Cause: %{public}s', JSON.stringify(er
          r) ?? '');
        return;
      }
      hilog.info(0x0000, 'testTag', 'Succeeded in loading the content.');
    });
  }
  onWindowStageDestroy(): void {
    // Main window is destroyed, release UI related resources
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onWindowStageDestroy');
  }
  onForeground(): void {
    // Ability has brought to foreground
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onForeground');
  }
  onBackground(): void {
    // Ability has back to background
    hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onBackground');
  }
}
```